Lab-6: CS-2203 Artificial Intelligence Lab

Assignment No: 06

Date: Feb 12, 2026

Submission Deadline: [Feb 12, 2026 - 17:50 Hrs. IST]

Instructions for Submission

General Instructions

Some assignments may need to be submitted via Google Form. Please carefully follow the instructions below to ensure proper submission:

File Preparation

Submission Instructions:

1. Store all assignment-related files in a single folder and compress it into a `.rar` or `.zip` file. The compressed file must be named in the following format:

   `roll-number_assign6.rar` or `roll-number_assign6.zip`

2. For example, if your roll number is `2201AI01`, the compressed file should be named as:

   `2201AI01_assign6.rar` or `2201AI01_assign6.zip`

3. Save each program using the filename format specified alongside each question.

4. If all assignments are completed within a single Jupyter Notebook, name the notebook file using the following format:

   `CS2203_Lab6.ipynb`

5. When using a single notebook for multiple problems, clearly **segregate the code for each problem**. Each problem must be introduced using a **text cell** containing the problem statement before the corresponding code segment.

6. After completing the notebook, place it inside the assignment folder, compress the folder, and upload it by following the above naming instructions.

Submission Link

Upload your compressed file using the following Google Form link:

`https://forms.gle/ZmDZ9go4pWbGpPUq8`

Problem Statement 1: Navigating the Maze with Depth-First Search (DFS) [20 + 10 points]

In this lab exercise, your goal is to use the **Depth-First Search (DFS)** algorithm to navigate through a maze represented as a grid. Each grid cell is either passable (0) or blocked (1). You will implement the DFS algorithm to explore the maze and find a path from a starting point to an end point.

The task requires you to implement DFS and handle both scenarios:

- A path exists from start to end.
- No path exists from start to end.

Task

- **Graph Representation:** The maze is represented as a 2D grid. Each cell is either a 0 (path) or 1 (wall). The starting point and the end point are provided as coordinates in the form (row, col).
- **DFS Algorithm:** Implement the DFS algorithm to traverse the maze. Use a backtracking approach to find a valid path from the start point to the end point.
- **Input:** The maze will be provided as a 2D list (matrix). You need to take this input within your Python code.
- **Output:**
  - If a path is found, output the path as a list of coordinates.
  - If no path is found, return `"No path found"`.

Example

Example 1: Path Exists

```
maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
end = (4, 4)

# Output: Path found: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2),
   (1, 2), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4,
   4)]
```

Example 2: No Path Exists

```
maze = [
    [0, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 0]
]

start = (0, 0)
end = (4, 4)

# Output: No path found
```

Input Handling Inside Code

To avoid time wastage during evaluation, you are required to take the maze grid, start point, and end point as inputs inside the code. The maze grid should be hardcoded within the script, and the start and end points should also be provided in the same code.

Implementation Details:

(a) You must use DFS to explore the maze.

(b) Ensure that the algorithm avoids revisiting cells by using a visited list.

(c) If DFS successfully finds the end point, output the sequence of coordinates from start to end.

(d) If DFS reaches a dead end and cannot find a path to the end, output `"No path found"`.

Sample Output 1:

```
Path found: [(0, 0), (1, 0), (2, 0), (-, -), (-, -)]
```

Sample Output 2:

```
No path found
```

Evaluation Criteria : 1

The evaluation of this lab assignment will be based on the following criteria:

(a) **Correctness of Implementation (40%):**

(b) **Code Quality & Q&A (30%):**

- The code should be well-structured, properly commented, and follow standard Python programming practices.
- Students may be asked questions related to their implementation to assess their understanding.

(c) **Error Handling (10%):**

- The code gracefully handles potential errors, such as file not found or invalid image formats.

(d) **Documentation (10%):**

- The code is accompanied by appropriate documentation and comments that explain the steps taken in each part of the program.

(e) **Timeliness of Submission (10%):**

- The assignment is submitted on time, adhering to the submission deadline.

Evaluation Criteria: 2

The evaluation of this lab assignment will be based on the following criterion:

(a) **Viva (10 Marks):**

- Questions will be asked related to the assignment to assess the student's understanding of the implemented concepts.